ECE 408

第 14 讲

Profiling on Nvidia GPU

目录

1 性能分析与 GPU 硬件 **1**
1.1 性能数据所回答的问题 . . . 1
1.2 线程组织与硬件执行层级 . . . 1
1.3 存储空间、物理位置与缓存路径 . . . 2

2 矩阵乘法基准与可信测量 **4**
2.1 统一问题规模与运算量口径 . . . 4
2.2 简单内核与共享内存分块 . . . 4
2.3 寄存器与共享内存联合分块 . . . 6
2.4 正确性检查与性能构建 . . . 7
2.5 流、事件与异步执行的计时范围 . . . 8

3 系统时间线与内核指标采集 **10**
3.1 在统一时间轴上定位耗时 . . . 10
3.2 系统级采集命令与报告 . . . 11
3.3 内核级采集、筛选与源码范围 . . . 12
3.4 采集扰动与可比较的报告 . . . 13

4 吞吐率、屋顶线与存储流量 **14**
4.1 理论峰值与资源利用指标 . . . 14
4.2 算术强度与屋顶线模型 . . . 14
4.3 矩阵乘法中不同层级的复用 . . . 15
4.4 从存储工作负载图读取实际数据流 . . . 16
4.5 全局访存合并与多余扇区 . . . 17
4.6 共享内存存储体与宽访问 . . . 18

5 线程束调度、占用率与指令定位 **19**
5.1 资源预算如何限制占用率 . . . 19
5.2 从驻留到就绪，再到实际发射 . . . 20
5.3 线程束停顿与延迟隐藏 . . . 21
5.4 从源代码行追踪到机器指令 . . . 22

6 矩阵乘法的比较与优化 **23**
6.1 三组设备上的测量结果 . . . 23
6.2 简单内核到共享分块：瓶颈如何改变 . . . 25
6.3 以向量读取缓解共享指令队列压力 . . . 26
6.4 由指标变化验证优化机制 . . . 27

1 性能分析与 GPU 硬件

1.1 性能数据所回答的问题

性能分析(*profiling*)是在程序实际执行时，收集运行时间、事件时间线和硬件计数器，再把这些观察与源代码联系起来的过程。**性能分析器**(*profiler*)可以观察处理器何时工作、数据经过哪些存储层级、指令为何等待，以及哪些代码占用了主要执行时间。程序的数学表达式给出了要完成的运算；执行性能还取决于线程组织、访存布局、资源分配和具体硬件。

优化需要先建立正确的程序，再测量其开销，并针对主要限制修改实现。每次修改之后都要重新检查结果和运行时间，因为减少一种开销可能增加另一种开销。例如，共享内存分块能减少重复的全局读取，同时也会引入共享读取、块内同步和片上资源占用；这些成本共同决定最终收益。

分析尺度可以分为应用和内核两层。应用层关心主机准备数据、设备传输、内核计算与同步之间的时间关系；内核层关心某次内核执行期间的指令发射、计算流水线、缓存及存储系统。系统级分析工具（Nsight Systems）适合先定位应用中的主要耗时区间，内核级分析工具（Nsight Compute）再用于解释目标内核内部的硬件行为。一次内核执行变快以后，还应回到应用层确认总时间是否下降。

设目标内核原先占应用总时间的比例为 f，对它的加速比为 s，其余部分的时间保持不变。将原总时间归一化为 1，优化后的时间为 $(1-f)+f/s$，所以应用加速比为

$$S_{\text{app}} = \frac{1}{(1-f)+f/s}. \tag{1}$$

例如，某内核只占总时间的 20%，即使它达到 4 倍加速，应用也只有 $1/(0.8+0.2/4) \approx 1.18$ 倍加速。这个关系说明了为什么应先确定耗时位置，再选择值得深入分析的内核。

1.2 线程组织与硬件执行层级

在统一计算设备架构（CUDA）的执行模型中，一次内核启动产生一个**网格**(*grid*)，网格由**线程块**(*thread block*)组成，线程块内包含线程。GPU 的主要通用执行单元是**流式多处理器**(*streaming multiprocessor*，*SM*)。一个普通线程块的线程在同一个 SM 上执行，因此能够通过该 SM 的共享内存交换数据并进行块内同步；一个 SM 在资源允许时可以同时驻留多个线程块。

线程块内部以 32 个线程组成一个**线程束**(*warp*)。线程束中的线程共享指令发射过程，各

线程仍保留自己的寄存器状态和数据索引。调度器从驻留线程束中选择已经具备执行条件的线程束，并向相应流水线发射指令。于是，线程束等待某次访存时，调度器可以发射其他就绪线程束的指令，这构成了**延迟隐藏**(*latency hiding*)的基础。

一个 SM 通常进一步划分为若干子分区（SM subpartition，SMSP），其中包含线程束调度器、寄存器资源和执行流水线。芯片级还可能采用图形处理簇（graphics processing cluster，GPC）、纹理处理簇（texture processing cluster，TPC）等组织层级。这些结构帮助解释硬件图，但程序中的线程块与这些簇没有固定的一一对应关系。线程块到 SM 的分配、每个 SM 可驻留的线程束数，以及每周期能够发射的指令，都要结合所分析的 GPU 判断。

不同指令会使用不同执行资源：普通浮点乘加使用浮点流水线，地址计算通常需要整数指令，读取与写入使用加载 / 存储流水线，某些数学函数使用特殊函数单元（special function unit，SFU）。张量核心（Tensor Core）承担特定矩阵运算。一个内核是否有效利用硬件，必须对应到它实际执行的指令类别；某种流水线很忙，无法直接推出全部计算资源都达到了峰值。

并行数量与执行能力

驻留线程束提供可供调度的工作，就绪线程束提供当前可发射的工作，执行流水线决定当前能接收什么指令。这三个层次共同限制吞吐率。增加线程数只有在改善其中的实际限制时才会带来收益。

1.3 存储空间、物理位置与缓存路径

CUDA 的存储空间规定数据的可见范围和访问方式；物理存储层级决定数据放在哪里、如何被缓存，以及访问经过哪些硬件。理解性能指标时，需要同时掌握这两种含义。表1给出与本讲相关的对应关系。

表 1 存储空间与典型硬件实现

存储空间	典型物理位置或访问路径	分析时的关注点
全局内存	设备内存，通常经过 L2 与 L1 / 纹理缓存路径	访问合并、缓存复用和各层实际流量
共享内存	SM 上显式管理的片上存储	线程块内复用、存储体冲突和读取指令压力
常量内存	设备内存及相应常量缓存路径	同一线程束访问相同地址时的广播
纹理 / 表面内存	设备内存及纹理相关缓存路径	专用访问方式及局部性
局部内存	线程私有地址空间，数据可能位于设备内存并经过缓存	大型私有数组和寄存器溢出带来的访存
寄存器	SM 的寄存器文件；按线程保存状态，并受分配粒度约束	寄存器用量、依赖链和可驻留线程束数

一级缓存（L1 cache）靠近 SM，二级缓存（L2 cache）位于更外层并由多个 SM 共享。许多架构让 L1 与共享内存共用一部分物理片上容量，但两者的管理方式不同：共享内存由程序显式写入、读取和同步，L1 则依照缓存机制自动保存访问过的数据。共享内存容量的配置可能影响可用缓存容量，也可能限制同时驻留的线程块数。

局部内存（*local memory*）中的“局部”表示线程私有的可见性。它不能作为低延迟片上存储的保证。当编译器无法把某些线程私有变量保留在寄存器中时，这些变量可能被放到局部内存，从而增加实际加载和存储。寄存器数量因此存在两方面作用：更多寄存器可以保存更多可复用数据和独立累加器，但每线程用量过高又可能压低占用率，甚至导致**寄存器溢出**（*register spilling*）。

缓存命中（*cache hit*）表示所需数据已经位于当前检查的缓存层；**缓存未命中**（*cache miss*）则需要继续向下一级请求数据。**时间局部性**（*temporal locality*）来自一段时间内反复访问同一数据，**空间局部性**（*spatial locality*）来自访问相邻地址。缓存容量有限，旧数据会因替换而被移出，因此同一个全局地址的多次访问不一定每次都需要访问设备内存，也不一定一直能够命中同一层缓存。

把一次加载简单理解为“L1 未命中后读取 L2，L2 未命中后读取设备内存”，能够解释数据来源；要进一步理解吞吐率，还必须考虑请求队列、地址标签检查、数据返回和执行依赖。图1展示了这种更具体的路径。共享请求与全局请求可以经过相关的入口和数据通路，因此即使设备内存流量不大，片上请求处理也可能形成限制。

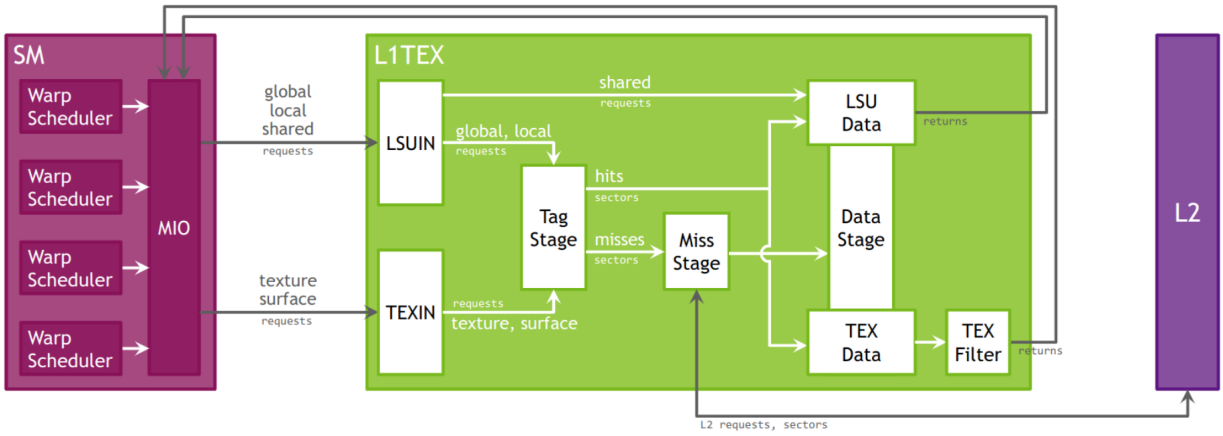


图 1 线程束调度器到 L1 / 纹理单元及 L2 的请求与返回路径。共享访问需要经过片上的请求处理和数据通路；全局访问还涉及缓存标签检查与未命中处理。

这些层级还解释了为何同一份 CUDA 代码在不同 GPU 上呈现不同的瓶颈。缓存容量、每个 SM 的执行资源、可驻留线程束数和设备内存带宽都可能改变。比较硬件时，应保留问题规模、数据类型、编译选项和计时范围，再依据对应设备的计数器解释差异。

2 矩阵乘法基准与可信测量

2.1 统一问题规模与运算量口径

通用矩阵乘法（general matrix multiplication，GEMM）提供了一个便于比较不同数据复用方式的基准。设 $A \in \mathbb{R}^{M\times K}$、$B \in \mathbb{R}^{K\times N}$，计算 $C = AB \in \mathbb{R}^{M\times N}$。采用从 0 开始的索引，则 $C_{ij} = \sum_{k=0}^{K-1} A_{ik}B_{kj}$。以下实现均使用**行主序**（*row-major order*）：A_{ik} 的元素偏移为 $iK + k$，B_{kj} 为 $kN + j$，C_{ij} 为 $iN + j$。代码假定 M, N, K 为正整数，设备数组容量与矩阵尺寸匹配。

每个输出需要 K 次乘法和累加。性能统计通常把一次**融合乘加**（*fused multiply-add*，*FMA*）计为两次浮点运算，即使它在机器层只对应一条指令。因此，统一采用总运算量 $F = 2MNK$。这里的浮点运算量（FLOP）与每秒浮点运算数（FLOP/s）含义不同：前者是工作量，后者是吞吐率。若耗时为 t_{ms} 毫秒，则

$$P_{\text{G}} = \frac{2MNK}{10^6 t_{\text{ms}}} \quad \text{GFLOP/s}, \qquad S = \frac{t_{\text{base}}}{t}, \qquad \eta = \frac{P}{P_{\text{peak}}}. \tag{2}$$

其中 S 是相对基准的加速比，η 是采用指定理论峰值计算的效率。峰值应与浮点精度、执行单元类型和时钟条件一致；用不同精度或张量核心的峰值评价普通单精度内核，会改变效率分母的含义。$2MNK$ 统计有效工作量；分块边界处执行的填零乘加也可能计入硬件计数，因此报告按实际指令统计的吞吐率与这里的有效吞吐率可能略有差异。

2.2 简单内核与共享内存分块

简单内核让每个线程负责一个输出。为了把输出空间排成一维网格，可令线程索引 q 对应 $i = \lfloor q/N \rfloor$、$j = q \bmod N$。这使相邻线程优先处理同一行的相邻列。下面的内核用 `size_t` 计算线性地址，并对输出边界进行检查。

代码 1 每线程计算一个输出的简单矩阵乘法

```
__global__ void gemm_simple(const float* A, const float* B,
                            float* C, int M, int N, int K) {
  size_t q = size_t(blockIdx.x) * blockDim.x + threadIdx.x;
  if (q >= size_t(M) * N) return;
  int row = int(q / N);
  int col = int(q % N);
  float sum = 0.0f;
  for (int k = 0; k < K; ++k) {
    sum += A[size_t(row) * K + k] * B[size_t(k) * N + col];
  }
  C[q] = sum;
}
```

从源代码看，每个输出执行 $2K$ 次全局加载。同一行的多个输出都要使用同一组 A 元素，同一列的多个输出也会重复使用 B 的元素。这些重复访问可能命中缓存，因此源代码加载次数并不直接等于设备内存事务次数。不过，程序仍然需要发射相关加载指令并处理地址与依赖，这些开销即使在缓存命中时也存在。

共享内存分块把 $T \times T$ 个输出交给一个包含 $T \times T$ 个线程的线程块。每次沿 K 方向处理长度为 T 的区间时，线程块先协作读入 A 的一个 $T \times T$ 子块和 B 的一个 $T \times T$ 子块，再让各线程从共享内存取数并更新自己的累加器。一个加载进来的 A 元素供同一输出行中的 T 个线程复用，一个 B 元素则供同一输出列中的 T 个线程复用。

对一个不含边界的完整分块阶段，简单实现需要约 $2T^3$ 次全局标量加载，共享分块只需 $2T^2$ 次；两者都完成 $2T^3$ 次浮点运算。因此，全局加载的逻辑需求约缩小到原来的 $1/T$。实现还需要共享内存加载和同步，其实际加速比应通过测量确定。

代码 2 带边界处理的共享内存分块内核

```
template<int T>
__global__ void gemm_shared(const float* A, const float* B,
                            float* C, int M, int N, int K) {
  __shared__ float As[T][T];
  __shared__ float Bs[T][T];
  int tx = threadIdx.x, ty = threadIdx.y;
  int row = blockIdx.y * T + ty;
  int col = blockIdx.x * T + tx;
  float sum = 0.0f;
  for (int k0 = 0; k0 < K; k0 += T) {
    As[ty][tx] = (row < M && k0 + tx < K)
        ? A[size_t(row) * K + k0 + tx] : 0.0f;
    Bs[ty][tx] = (k0 + ty < K && col < N)
        ? B[size_t(k0 + ty) * N + col] : 0.0f;
    __syncthreads();
    for (int k = 0; k < T; ++k) {
      sum += As[ty][k] * Bs[k][tx];
    }
    __syncthreads();
  }
  if (row < M && col < N) C[size_t(row) * N + col] = sum;
}
```

代码2要求以 `dim3(T,T)` 作为线程块维度，以 `dim3((N+T-1)/T,(M+T-1)/T)` 作为网格维度。合法的 T 还须满足设备对线程块线程数和共享内存的限制。第一个同步保证整个子块已经加载完成；第二个同步保证所有线程读取完当前子块后，才会有人覆盖下一阶段的数据。边界线程仍参与加载、补零与两个同步，不能在进入循环之前因输出越界而提前返回。

这一实现也给出了两种不同的连续性。全局加载时，相邻 `threadIdx.x` 访问 A 或 B 的相邻列；累加时，固定线程沿 k 依次读取 `As[ty][k]`，这些地址连续，而 `Bs[k][tx]` 的相邻读取之间相隔一整行。前一种连续性关系到线程束的全局访存合并，后一种连续性决定单个线程是否方便把多次共享读取合成宽指令。

2.3 寄存器与共享内存联合分块

线程粗化(*thread coarsening*) 让一个线程负责多个输出。设一个线程计算某行上的 r 个相邻输出，则它需要 r 个累加器 $c_0, \dots, c_{r-1}$。对某个归约索引 k，只需把 $a = A_{ik}$ 加载到一个寄存器中，就能用它更新 r 个累加器：$c_j \leftarrow c_j + aB_{k,j_0+j}$。$A$ 的这次加载于是在线程内部得到复用。

联合分块可以让一个线程块的 b_m 个线程分别处理 b_m 行，每线程负责 b_n 列输出，形成 $b_m \times b_n$ 的输出子块。各线程使用的 A 行不同，适合把当前 A 数据保留在线程自己的寄存器

中；所有线程会重复使用同一组 B 数据，适合将 B 子块放入共享内存。这将复用分别安排在线程内部和线程块内部。

若每个归约阶段的长度为 b_k，理想情况下需要加载 $b_m b_k$ 个 A 元素和 $b_k b_n$ 个 B 元素，完成 $2b_m b_n b_k$ 次浮点运算。线程同时持有多个输出累加器，还能提供多条相对独立的乘加依赖链。代价是寄存器需求增大、线程块配置改变，边界处理和数据加载的分工也更复杂。合理的分块形状需要兼顾复用、访存布局与资源限制。

高性能线性代数库（cuBLAS）提供 GEMM 的优化实现，可以作为参考结果和性能对照。比较时应统一矩阵尺寸、行列主序的解释、输入与累加精度，以及是否允许采用张量核心。以下实验表采用普通单精度计算口径，不将张量核心峰值混入效率分母。库函数的函数名本身不足以确定全部执行路径，数学模式和实际指令也属于实验配置。

2.4 正确性检查与性能构建

性能比较成立的前提是各实现完成同一个计算。对矩阵乘法，可以先使用可手工核算的小矩阵，再使用中央处理器（CPU）参考实现或可信的 GPU 实现比较全部输出。全 1 输入能快速检查数量级，因为每个输出都应等于 K；非方阵、非均匀数据和不能被分块宽度整除的尺寸则能检查索引与边界。仅检查少数输出可能漏掉线程映射错误。

浮点加法不满足严格结合律，不同分块或乘加融合方式会改变舍入路径。设参考结果为 C_{ij}^{ref}，一种逐元素判据是

$$|C_{ij} - C_{ij}^{\mathrm{ref}}| \le \varepsilon_{\mathrm{abs}} + \varepsilon_{\mathrm{rel}} |C_{ij}^{\mathrm{ref}}|. \tag{3}$$

绝对容差照顾接近零的结果，相对容差照顾数值尺度较大的结果。容差应依据数据类型、输入尺度和累加长度选定，同时单独检查非有限值；不能因为更宽松的容差能通过，就认定一个越界或竞争问题已经解决。

计算检查器（Compute Sanitizer）提供几种互补检查。`memcheck` 检查非法或未对齐的内存访问，并可检查泄漏；`racecheck` 检查共享内存访问竞争；`initcheck` 检查未初始化的全局内存读取；`synccheck` 检查同步原语的使用错误。对代码2，缺少前一个同步可能让线程读取尚未写入的共享元素，缺少后一个同步则可能使下一阶段覆盖仍在被读取的数据。

代码 3 构建待测程序并分别运行正确性工具

```
nvcc -O3 -lineinfo gemm_demo.cu -o gemm_demo
compute-sanitizer --tool memcheck ./gemm_demo
compute-sanitizer --tool racecheck ./gemm_demo
compute-sanitizer --tool initcheck ./gemm_demo
compute-sanitizer --tool synccheck ./gemm_demo
```

这里的 `gemm_demo.cu` 和 `gemm_demo` 分别表示待测程序的源文件与可执行文件；构

建时还应选择与目标 GPU 匹配的架构。检查工具会改变运行开销，它们的执行时间不用于比较正常性能。不同工具通过以后，仍要确认数值输出，因为内存访问合法并不能保证公式或索引正确。

性能构建应保留正常优化，并加入 `-lineinfo`，以便把设备指令映射回源代码行。表2区分容易混用的选项。

表 2 与性能分析相关的编译选项

选项	作用	对性能分析的意义
`-O3`	启用主机编译的优化级别	与正常发布构建保持一致；设备调试选项需另行检查
`-lineinfo`	生成设备代码行号信息	便于把热点定位到源代码，并保留性能优化
`-g`	生成主机调试信息	本身不等于关闭设备代码优化；不是设备行号信息的替代品
`-G`	生成设备调试信息	通常会关闭设备优化，适合调试；一般性能测量避免使用
`-pg`	为传统主机分析器生成插桩	不属于此处的 GPU 性能分析构建选项

2.5 流、事件与异步执行的计时范围

CUDA **流**（*stream*）是按顺序执行操作的队列，类型为 `cudaStream_t`。同一流中的内核、传输和事件遵循该流的顺序；不同流之间可以在资源和依赖允许时重叠。未显式给出流时使用默认流，其跨流同步语义还受默认流模式影响。使用明确创建的流，能让计时范围及依赖更容易检查。

一次内核启动通常先将工作提交给 GPU，主机随后继续执行。因此，主机在内核启动语句前后立即读取时钟，可能主要测到提交开销。CUDA **事件**（*event*）是插入某个流中的标记，类型为 `cudaEvent_t`；事件时间戳在设备执行推进到该标记时记录。同一流中依次插入开始事件、待测内核和结束事件，再等待结束事件完成，就能测量这一段设备时间。

以下函数接受一个 `launch` 可调用对象，它将待测内核启动到参数指定的同一条流中。头文件提供运行时类型和异常类型；`check` 把失败的 CUDA 调用转成异常，避免把未正常执行的内核计入有效结果。

代码 4 带预热与重复执行的 CUDA 事件计时

```cpp
#include <cuda_runtime.h>
#include <stdexcept>

inline void check(cudaError_t status) {
  if (status != cudaSuccess)
    throw std::runtime_error(cudaGetErrorString(status));
}

template<class Launch>
float average_kernel_ms(Launch launch, cudaStream_t stream,
                        int warmup, int repeats) {
  if (warmup < 0 || repeats <= 0)
    throw std::invalid_argument("invalid iteration count");
  for (int i = 0; i < warmup; ++i) launch();
  check(cudaGetLastError());
  check(cudaStreamSynchronize(stream));
  cudaEvent_t start, stop;
  check(cudaEventCreate(&start));
  check(cudaEventCreate(&stop));
  check(cudaEventRecord(start, stream));
  for (int i = 0; i < repeats; ++i) launch();
  check(cudaGetLastError());
  check(cudaEventRecord(stop, stream));
  check(cudaEventSynchronize(stop));
  float elapsed_ms = 0.0f;
  check(cudaEventElapsedTime(&elapsed_ms, start, stop));
  check(cudaEventDestroy(start));
  check(cudaEventDestroy(stop));
  return elapsed_ms / repeats;
}
```

例如，若 A、B、C 已经指向设备数组，并已创建 `stream`，可使用以下调用。这里的 5 次预热和 20 次重复只是计时示例参数。

代码 5 将计时对象显式绑定到同一条流

```cpp
dim3 block(16, 16);
dim3 grid((N + 15) / 16, (M + 15) / 16);
auto launch = [&]() {
  gemm_shared<16><<<grid, block, 0, stream>>>(
      A, B, C, M, N, K);
};
float ms = average_kernel_ms(launch, stream, 5, 20);
```

分配、数据上传和结果校验位于计时区间之外，就得到内核计算口径；若要比较应用端到端时间，则需要另行包含这些工作，并在主机计时结束前等待对应的设备工作完成。

预热可以排除首次上下文初始化、延迟加载及起始时钟状态的一部分影响，但预热也改变缓存状态。重复运行同一矩阵得到的是该重复执行场景的性能，应与实际应用的输入复用

方式相符。对很短的内核，两事件间还可能存在主机未及时提交后续启动导致的空隙；因而重复平均时间仍需结合时间线理解。

流间可以通过事件记录函数 `cudaEventRecord` 与流等待函数 `cudaStreamWaitEvent` 建立“生产完成后再消费”的依赖。先在生产流记录完成事件，再让消费流等待该事件，就只建立了所需的先后关系，并不要求主机一直阻塞。若要测量多条流共同完成的总设备区间，需要让结束标记依赖所有相关流的完成事件；随意在两条无依赖关系的流上分别记录开始和结束事件，不能定义可靠的整体工作范围。

主机与设备传输能否和计算重叠，还取决于复制引擎、数据依赖、流配置，以及主机缓冲区是否适合异步传输。多条流提供表达并发的机会，实际是否重叠必须在时间线上确认。相同原则也适用于频率：记录 GPU、工具版本、时钟策略和测量范围，才能解释不同测试之间的差异。

3 系统时间线与内核指标采集

3.1 在统一时间轴上定位耗时

系统级分析将 CPU 活动、CUDA 应用程序接口（application programming interface，API）调用、GPU 内核和数据传输标在统一的时间轴上。它能区分几种表面上都表现为“程序很慢”的情况：主机尚未提交工作，GPU 正在搬运数据，目标内核本身运行较久，或者某个同步点正在等待前面的工作完成。统计视图给出调用次数、累计时间、平均时间等信息，时间线则展示这些事件之间的重叠和先后关系。

图2中，主机上的流同步调用 `cudaStreamSynchronize` 与 GPU 上连续执行的矩阵乘法重叠。该调用的长区间表示主机在等待指定流完成；沿同一时间轴观察 GPU，才能判断等待期间发生了什么。若直接把主机同步累计时间与 GPU 内核累计时间相加，同一段真实经过的时间就可能被重复计算。

分析时可以先观察最长区间，再检查其中的结构。若多个短内核之间有明显空隙，应查看主机提交、同步和数据准备；若一段传输之后才开始计算，应检查依赖是否允许重叠；若单个内核占据主要时间，就适合进入内核级分析。内存分配也可能是显著的一次性开销，反复测试时应明确它是否属于目标应用的正常路径。

厂商工具扩展接口（NVIDIA Tools Extension，NVTX）允许程序为主机代码区间命名。命名区间能把“初始化”“矩阵乘法”“结果校验”等阶段对应到时间线。主机区间的结束通常只说明该段主机代码已执行完；如果其中只提交了异步 GPU 工作，设备执行可能延续到区间之后。命名区间、GPU 上对应活动及同步事件需要共同解释。

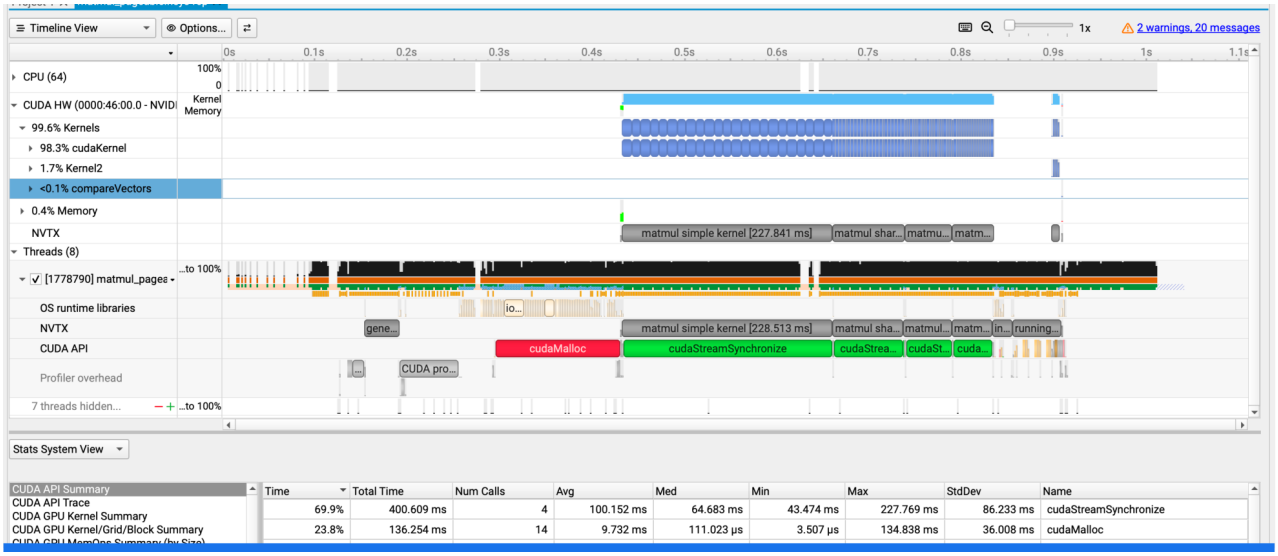


图 2 主机、CUDA 接口和 GPU 活动的统一时间线。同步函数在主机侧持续等待的同时，设备侧可以连续执行多个内核；区间长度需要结合所属轨道解释。

3.2 系统级采集命令与报告

远端采集与本地查看可以分开完成：在具有目标 GPU 的机器上运行命令行采集器 `nsys`，生成 `.nsys-rep` 报告，再用图形客户端 `nsys-ui` 打开。查看已有报告不需要在本地重新执行同一个 GPU 内核。一次普通采集可写成：

代码 6 系统级采集与报告查看

```
nsys profile --trace=cuda,nvtx,osrt \
  --output=gemm_system --force-overwrite=true ./gemm_demo
nsys-ui gemm_system.nsys-rep
```

其中 `profile` 启动待测程序并按指定范围采集。`--trace` 选择 CUDA、命名区间和操作系统运行时等事件；`--output` 指定报告文件名的主体；`--force-overwrite=true` 允许覆盖同名报告。命令行中的续行符必须位于物理行末，后面不应再接注释。

表 3 系统级采集的常用控制选项

选项	用途
`-o`、`--output`	设置输出报告名称
`-f`、`--force-overwrite`	控制是否覆盖已有报告
`-e`、`--env-var`	为被分析程序设置环境变量
`-y`、`--delay`	延迟指定秒数后开始采集，适合略过已知的启动阶段
`--capture-range=nvtx`	用指定的 NVTX 区间控制采集范围
`-p`、`--nvtx-capture`	选择 NVTX 区间；需要同时启用相应范围控制
`--stats=true`	采集后生成统计摘要；数据库导出和统计会增加分析时间

对较复杂的程序，交互模式把启动和采样控制分开：`launch` 在适合分析的环境中启动

程序，start 与 stop 控制采样开始和结束，shutdown 结束分析会话。提供 cancel 的版本可以用它取消采集并丢弃尚未保留的数据。停止采样、结束会话和终止目标进程属于不同动作，目标进程的后续状态还受会话配置影响。某些 CUDA 事件需要在程序启动时注入分析支持，不能假设对任意已运行进程开始采样都能补齐此前的接口跟踪。

3.3 内核级采集、筛选与源码范围

内核级分析以一次或若干次选定的内核启动为对象，读取相关硬件计数器，并组织成吞吐率、存储工作负载、调度、占用率及源码热点等报告。命令行工具 ncu 直接接收选项和目标程序，不采用 nsys profile 那样的子命令形式。其报告扩展名为 .ncu-rep，图形客户端为 ncu-ui。

代码 7 筛选预热之后的一次目标内核启动

```
ncu --set full --kernel-name regex:gemm_shared \
  --launch-skip 5 --launch-count 1 \
  --export gemm_kernel --force-overwrite ./gemm_demo
ncu-ui gemm_kernel.ncu-rep
```

这个例子先按内核名称筛选，再跳过前 5 次匹配的启动，只分析接下来 1 次匹配启动。若被测程序使用其他内核名称，就应相应替换过滤表达式。跳过次数按匹配结果计数；若程序中穿插了初始化或校验内核，它们不会仅因位置较早就自动消耗这里的匹配次数。

表 4 内核级采集的常用选项

选项	用途
-o、--export	导出报告；-f 允许覆盖已有输出
--set basic、--set full	选择基本指标或完整指标集；--list-sets 列出可用集合
-k、--kernel-name	按内核名称或匹配表达式筛选
--kernel-id	用上下文、流、内核名称或启动编号等条件精确定位
-s、--launch-skip	跳过指定数量的匹配启动
-c、--launch-count	限制采集的匹配启动次数
--profile-from-start off	由程序中的性能分析控制接口决定开始与结束
--nvtx-include	配合 --nvtx，只分析匹配 NVTX 区间内启动的内核

程序也可以显式圈定采集范围。下面的函数接收与计时函数相同形式的启动对象，要求它把内核提交到给定流，并假定目标上下文中没有其他流并发提交工作。先排空指定流此前的工作，再开启采样、执行目标并等待其完成，最后关闭采样，使范围边界清楚。控制接口作用于当前上下文，因此若有其他流在范围内提交内核，这些工作也可能被采集。

代码 8 以运行时接口圈定目标内核

```cpp
#include <cuda_profiler_api.h>

template<class Launch>
void profile_one(Launch launch, cudaStream_t stream) {
  check(cudaStreamSynchronize(stream));
  check(cudaProfilerStart());
  launch();
  check(cudaGetLastError());
  check(cudaStreamSynchronize(stream));
  check(cudaProfilerStop());
}
```

代码 9 配合程序内部范围进行采集

```
ncu --profile-from-start off --set full \
  --export gemm_range ./gemm_demo
```

代码8复用代码4中的错误检查函数。启动与停止接口分别为 `cudaProfilerStart` 和 `cudaProfilerStop`。若命令在可执行文件名之后出现 `--profile` 一类参数，该参数属于待测程序自身，只有程序显式实现了相应逻辑才有效；它与分析器的 `--profile-from-start` 属于不同参数层级。

3.4 采集扰动与可比较的报告

硬件能够同时读取的计数器数量有限，完整指标集往往需要**重放**(*replay*)：分析器将一次内核启动划分为若干采集轮次，在必要时保存和恢复相关内存状态。采集还可能改变缓存状态、时钟控制和原本可重叠的内核执行关系。因此，整条 `ncu` 命令的墙钟耗时包含采集与数据处理开销，不能拿来充当正常的单次内核时间。

正常 CUDA 事件计时回答“在给定执行场景下需要多久”，分析报告则提供“为何如此”的硬件证据。比较不同内核的报告时，应尽量固定输入、启动次数、指标集、缓存与时钟控制，并选择相同阶段。开启完整指标前，可以先采集基本报告确定目标；待有明确假设后再补充相应细节，以减少无关数据和重放次数。

图形界面的**基准比较**(*baseline comparison*)可以把一个报告中的目标内核设为参考，并展示另一内核各项指标的相对变化。若基准值为 x_0、新值为 x_1，相对变化为 $(x_1 - x_0)/x_0$。命中率从 98% 降到 52% 是降低 46 个百分点，相对下降约 46.9%；报告显示的相对百分比应与这两种口径区分。对于基准中原本为零的指标，相对百分比通常没有有用含义，应直接查看新出现的绝对数值。

4 吞吐率、屋顶线与存储流量

4.1 理论峰值与资源利用指标

硬件的理论吞吐能力常被称为峰值上限（Speed of Light，SoL）。内核报告中的计算和存储汇总指标，用相对于对应资源峰值的比例描述当前活动强度。由于一个汇总项包含多个组成资源，应先检查它由哪些计数器形成，再判断接近饱和的是浮点流水线、加载 / 存储流水线、缓存入口还是设备内存接口。

设某个汇总指标的组成资源利用比例为 $u_1, \dots, u_q$，相关吞吐指标采用最大组成比例，即 $u_{\text{group}} = \max_j u_j$。例如，加载 / 存储通路达到约 98%，而浮点流水线只有较低活动度，汇总项仍可能接近 98%。它描述的是最忙组成资源的状态，不能直接用来计算全芯片的总体浮点效率。

这与式(2)的效率 $\eta = P/P_{\text{peak}}$ 有不同的用途。η 由选定的浮点工作量和耗时计算，回答目标运算相对于对应算力峰值达到了多少；资源汇总及其分解则帮助判断什么硬件正在承受最大压力。若两者相差很大，通常应继续检查计数口径、精度、时钟，以及非算术指令的开销。

4.2 算术强度与屋顶线模型

算术强度（*arithmetic intensity*）表示每传输一个字节所完成的浮点运算量。设某内核工作量为 F，在指定存储层级 ℓ 实际读取和写入的总字节数为 Q_ℓ，则 $I_\ell = F/Q_\ell$，单位为 FLOP/byte。存储层级必须明确：设备内存的读写量、L2 与 L1 之间的流量、共享内存访问量，各自对应不同的强度和带宽上限。

先考虑一个只有计算峰值 P_{peak} 与指定层级带宽峰值 B_ℓ 的模型。完成全部计算至少需要 F/P_{peak} 时间，完成全部数据传输至少需要 Q_ℓ/B_ℓ 时间。即使二者理想重叠，总时间也不能小于其中的较大者。因此

$$t \geq \max\left(\frac{F}{P_{\text{peak}}}, \frac{Q_\ell}{B_\ell}\right) \quad \Longrightarrow \quad P = \frac{F}{t} \leq \min\left(P_{\text{peak}}, B_\ell I_\ell\right). \tag{4}$$

这个上界称为**屋顶线模型**（*roofline model*）。在双对数坐标下，$B_\ell I_\ell$ 是斜线，P_{peak} 是水平线；二者相交处的算术强度为 $I_\ell^* = P_{\text{peak}}/B_\ell$，也称转折点。

当 $I_\ell < I_\ell^*$ 时，带宽项更小；当 $I_\ell > I_\ell^*$ 时，计算项更小。在内核具有足够并行度、能够充分驱动相关资源的条件下，二者分别对应**带宽受限**（*memory-bound*）和**计算受限**（*compute-bound*）的典型区域。若测量点离两条上界都很远，则需要进一步查找延迟、依赖、指令发射

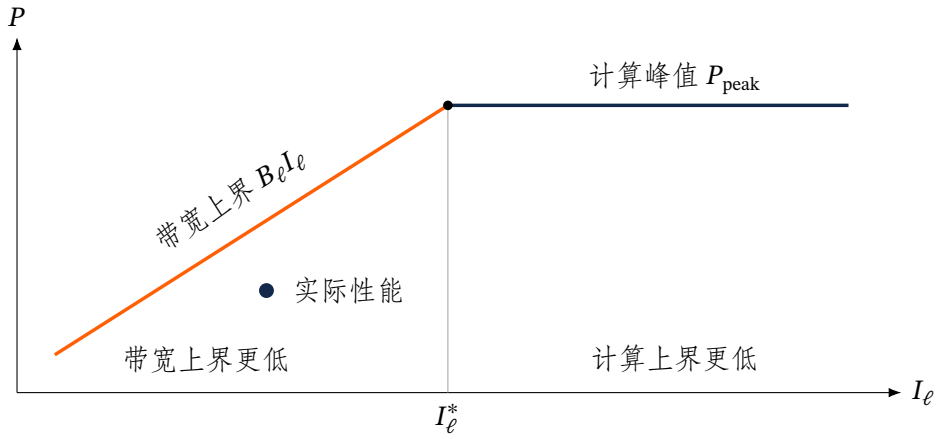


图 3 屋顶线模型的双对数示意。上界由算力和指定层级带宽中更严格的一项确定，实际内核还可能受到延迟、调度或指令通路限制。

或其他片上通路的限制；仅比较横坐标与转折点不足以给出完整诊断。

例如，设某 GPU 对当前精度的峰值为 20 TFLOP/s，对应层级带宽为 1 TB/s，则转折点为 20 FLOP/byte。算术强度为 4 的内核，其模型上界为 4 TFLOP/s；若实测只有 0.5 TFLOP/s，就仅达到该上界的 12.5%，还需检查为什么带宽没有被有效使用。若强度提高到 80，带宽项达到 80 TFLOP/s，模型上界则由 20 TFLOP/s 的计算峰值决定。这些数字用于说明计算步骤，并非某一实验 GPU 的规格。

不同 GPU 的算力和带宽比例不同，同一个内核的横坐标也可能因缓存行为改变。因此，换用计算峰值更高的 GPU 后，内核可能更接近带宽限制；反过来，带宽提高较多的设备也可能让计算上界成为更严格的约束。使用多个精度的屋顶线时，还需要选择与实际执行精度相符的水平峰值线。

4.3 矩阵乘法中不同层级的复用

算术强度的推导必须说明数据流量假设。对单精度 $C = AB$，若 A 和 B 各从设备内存读入一次，C 写回一次，则理想流量为 $Q_{\text{once}} = 4(MK + KN + MN)$ 字节，理想强度为

$$I_{\text{once}} = \frac{2MNK}{4(MK + KN + MN)}. \tag{5}$$

对边长为 n 的方阵，这给出 $I_{\text{once}} = n/6$。这个结果描述一次读入、一次写回的理想流量；真实分块中，不同线程块可能重复读取输入子块，能否由缓存承接这些重复访问取决于实际执行。

为了比较源代码层面的加载需求，可暂时忽略缓存，并令 n 能被分块宽度 T 整除。简单

内核有约 $2n^3$ 次单精度输入加载和 n^2 次输出写入，因此逻辑字节数约为 $8n^3+4n^2$，强度在 n 较大时趋近 1/4。共享分块将输入加载减少到约 $2n^3/T$ 个元素，逻辑字节数变为 $8n^3/T+4n^2$，相应强度趋近 $T/4$。例如 $T=16$ 时，这个逻辑加载模型给出约 4 FLOP/byte。

这里的 1/4、$T/4$ 与 $n/6$ 采用了不同的数据流量假设。前两者比较内核发出的逻辑全局访问需求，后者是理想的一次读写边界。若使用设备内存屋顶线，就应以实测设备内存字节数计算 I_{DRAM}；缓存命中的重复加载不会每次都计入这部分字节。因此，简单内核在实测设备内存屋顶线中的强度完全可能高于共享内核，同时运行得更慢。

在带宽上界主导且其他资源仍有余量时，提高每份已加载数据上的有效计算量，例如复用数据或融合需要相同输入的运算，可以提高强度。新增计算仍要满足寄存器、同步和指令通路的限制；人为加入无用运算，只会改变统计工作量，不能构成有意义的应用加速。对已经接近计算峰值的内核，则应考虑减少必要运算、改善指令组织，或在精度要求允许时使用更适合的计算路径。低精度路径的峰值提升幅度取决于具体架构和指令，不采用固定倍数作为通用结论。

4.4 从存储工作负载图读取实际数据流

存储工作负载图（Memory Workload Analysis）把左侧的逻辑存储空间与右侧的物理缓存层级联系起来。图4中，左侧记录指令活动，中间记录向存储单元发出的请求，右侧箭头则显示 L1、L2 与设备内存之间的数据流量。不同位置的数字描述不同对象，不能因为都与“内存访问”有关就直接相互替代。

指令（*instruction*）是实际执行的加载或存储操作；**请求**（*request*）是指令执行时向对应硬件单元提出的访问需求；**扇区**（*sector*）是缓存传输的计量粒度；**波前**（*wavefront*）表示硬件为了完成请求而进行的一轮服务。一个线程束的访存指令可以涉及多个地址、多个扇区或多个服务波前。报告中不少指令和请求指标以线程束为计数单位，而字节流量按实际传输计数，使用时还应核对单位。

在本讲的 L1 / L2 访存分析中，一个扇区为 32 字节。若同一请求的 32 个线程分别读取 4 字节，全部地址连续且与 32 字节边界对齐，则共需 128 字节，对应 4 个扇区。地址分散时，硬件可能要搬入更多扇区，虽然线程最终使用的数据量没有增加。部分线程不活跃、广播或每线程访问宽度不同时，理想扇区数也会随之改变。

缓存命中率回答按指定统计单位计算的访问命中比例；本图中的 L1 / L2 命中率按扇区统计，不能与请求命中率混用。吞吐率回答单位时间处理了多少数据或请求。高命中率与低延迟通常有利，但大量命中请求仍可能让片上入口、加载流水线或请求队列饱和。反过来，一次优化减少了请求数，却把访问顺序改变得更不利于复用，外层流量也可能上升。判断效

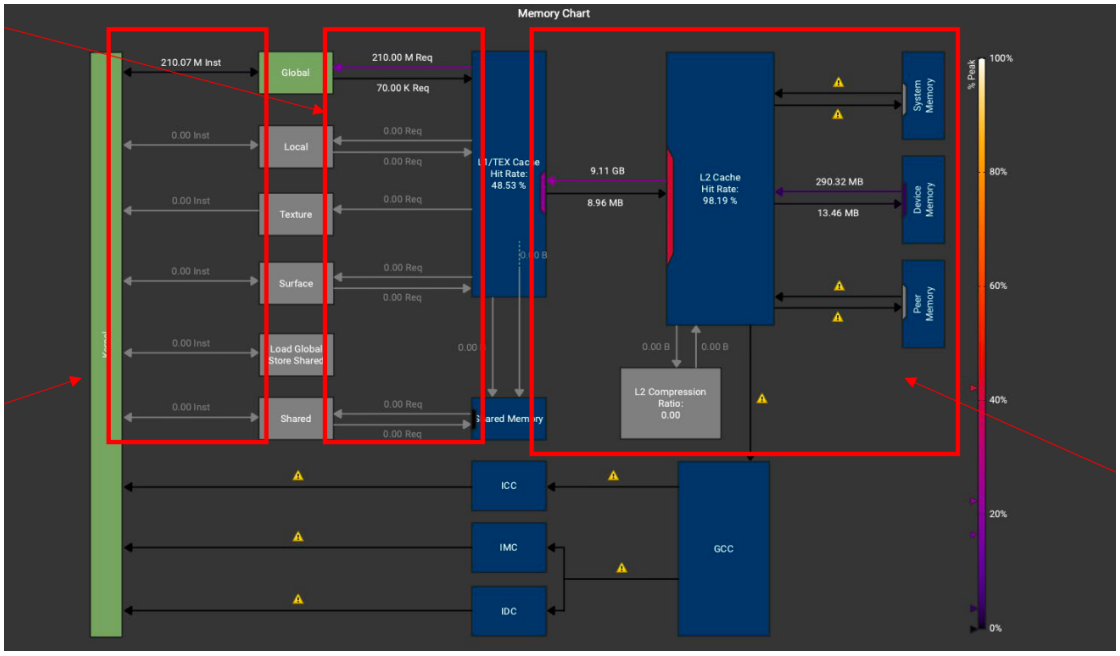


图 4 存储工作负载中的指令、请求与字节流量。读取可以在不同缓存层级得到满足，缓存命中与读写策略共同决定层级之间的流量。

果需要联合检查指令数、请求数、每请求扇区数、各层命中率和运行时间。

4.5 全局访存合并与多余扇区

合并访存（*coalesced memory access*）利用同一线程束各线程地址之间的空间关系，尽可能用较少的存储事务服务访问。表5以 32 个活跃线程、每线程读取一个单精度数为例，令 $j = 0, \dots, 31$，a_j 为第 j 个线程的字节地址，并假定基地址为 0。各线程读取的数据互不重叠，实际需要的有用字节总数均为 128。

表 5 地址布局对所需扇区数量的影响

地址形式	覆盖扇区数	扇区总字节	解释
$a_j = 4j$	4	128	连续且对齐，全部传输字节均有用
$a_j = 4j + 4$	5	160	连续访问跨过额外扇区边界
$a_j = 8j$	8	256	每隔一个单精度元素读取，利用一半字节
$a_j = 32j$	32	1024	每线程落入不同扇区，只使用其中 4 字节

这个例子中的扇区利用比例可写为 $\eta_{\text{sector}} = 128/(32n_s)$，其中 n_s 是实际覆盖的扇区数，四种布局分别得到 100%、80%、50% 和 12.5%。这些是对应请求的传输效率；后续缓存命中可能再次改变设备内存实际流量，因此不能把该比例直接当成整机带宽效率。

多余扇区指标把某种理想访问需求与实际需要的扇区比较，用于定位可能存在的合并不

足。它指出值得检查的代码位置，随后仍需结合每线程字节数、活跃线程数、对齐与请求类型解释。通常可以通过让相邻线程访问相邻元素、调整线程到输出的映射、修正行跨度或采用更合适的加载分工来改善。

对二维线程块，还应记住线程线性编号优先沿 x 方向增长。例如 16×16 的线程块中，一个线程束跨越两行、每行 16 个线程；是否高效取决于这两个半行分别覆盖多少扇区，以及行起点是否对齐。线程束跨两行本身不会必然导致浪费，必须代入实际地址分析。

4.6 共享内存存储体与宽访问

共享内存按**存储体**(*bank*)组织。在本讲使用的基本模型中，共有 32 个存储体，连续的 32 位字映射到连续存储体，每个存储体每周期提供一个 32 位字的带宽。若字节地址为 a，则存储体编号为 $b(a)=\lfloor a/4\rfloor \bmod 32$。访问无需按地址连续排列；只要同一次服务中的不同地址能分散到不同存储体，就可以并行服务。

存储体冲突(*bank conflict*)发生在同一服务阶段中，多个不同的 32 位字映射到同一存储体，导致额外串行服务。多个线程读取同一个 32 位字时，可以由一次读取及广播满足，包括该字内较窄的读取；这与读取同一存储体中的不同字有不同的代价。同地址并发写入则还涉及数据竞争，不能用读取广播来说明其正确性。

考虑完整线程束中的标量读取，每线程访问字索引 $w_j=w_0+sj$，其中步长 s 是正整数，32 个地址互不相同。存储体编号为 $(w_0+sj) \bmod 32$。若两个线程间隔 Δj 后落入同一存储体，就有 $s\Delta j\equiv 0 \pmod{32}$。令 $d=\gcd(s,32)$，最小正间隔是 $32/d$，所以访问覆盖 $32/d$ 个存储体，每个存储体需要服务 d 个不同地址。在这个标量模型下，冲突程度就是 d。

因此，$s=1$ 访问全部 32 个存储体，没有额外冲突；$s=2$ 只访问 16 个存储体，形成两路冲突；$s=32$ 的 32 个地址都落入同一存储体，形成 32 路冲突；$s=33$ 又能覆盖全部存储体。二维共享数组宽度为 32 时，沿列读取对应步长 32；将物理行宽填充到 33，就能让相邻行的同一列落入不同存储体。这个推导针对标量、互异地址；同地址广播和宽访问需要分别处理。

宽共享读取还要求区分每线程宽度和整个线程束的数据量。一条 `LDS.128` 指令中的 128 表示每线程 128 位，即 16 字节，可以装入四个单精度数。若 32 个线程读取互不相同的连续 16 字节区域，总请求为 512 字节；在常见的每轮 128 字节服务模型下，需要四个理想波前。这些必需的波前属于访问宽度带来的工作量，不能仅因同一存储体在不同波前中重复出现就判为冲突。

共享访问的多余波前指标考察实际服务波前超过理想需求的部分，能够帮助定位冲突。向量化可能减少指令和请求数量，同时保留相同的传输字节数；若布局和对齐合适，它就能

缓解指令队列压力。至于具体共享读取是否被编译为宽指令，以及额外波前是否下降，还需要结合机器指令与报告验证。

5 线程束调度、占用率与指令定位

5.1 资源预算如何限制占用率

占用率(*occupancy*)描述一个 SM 中驻留线程束数相对于硬件最大线程束数的比例。**理论占用率**(*theoretical occupancy*)由内核资源需求和启动配置限制，**实际占用率**(*achieved occupancy*)则由执行过程中测得的平均驻留情况得到。驻留表示线程束已经占有相应执行状态，并不要求它当前已经就绪。

设每个线程块有 T_b 个线程，对应 $w_b = \lceil T_b/32 \rceil$ 个线程束；一个 SM 允许的最大线程数、线程束数和线程块数分别为 $T_{\max}$、$W_{\max}$、$B_{\max}$。设寄存器容量为 R_{SM}，共享内存容量为 S_{SM}，每块实际占用量为 R_b 和 S_b，包含需要计入的驱动保留与分配取整。在简化资源预算中，可驻留线程块数 B_{res} 满足

$$B_{\mathrm{res}} \le \min\left\{B_{\max}, \left\lfloor \frac{T_{\max}}{T_b} \right\rfloor, \left\lfloor \frac{W_{\max}}{w_b} \right\rfloor, \left\lfloor \frac{R_{\mathrm{SM}}}{R_b} \right\rfloor, \left\lfloor \frac{S_{\mathrm{SM}}}{S_b} \right\rfloor\right\}, \qquad O_{\mathrm{theory}} = \frac{B_{\mathrm{res}} w_b}{W_{\max}}. \tag{6}$$

当实际分配量 $S_b = 0$ 时，应略去共享内存这一限制项。寄存器需求不能总是直接用每线程寄存器数乘线程数，因为真实分配会按线程束、线程块或子分区等硬件粒度取整；更详细的资源限制还要由目标设备的占用率报告确认。

考虑一个明确的预算例子：每个 SM 最多驻留 1536 个线程、48 个线程束和 16 个线程块，具有 65536 个 32 位寄存器及 100 KiB 可用共享内存。内核每块 256 个线程，即 8 个线程束；每线程需要 32 个寄存器，共享内存预算中的每块总占用量为 16 KiB。暂时忽略寄存器的额外分配取整，则每块占 8192 个寄存器，五个限制依次为 $16, 6, 6, 8, 6$，最多可驻留 6 块，对应 48 个线程束和 100% 理论占用率。

若只把寄存器用量提高到每线程 64 个，每块需要 16384 个寄存器，寄存器上限就变成 4 块，于是最多驻留 32 个线程束，占用率降至 $32/48 \approx 66.7\%$。这并不能单独判断粗化后的内核是否更慢：新增寄存器也可能减少重复读取、保存更多独立累加器，从而改善每个线程束的执行效率。这个预算例子中的硬件数量用于演示计算，不作为所有 GPU 的通用规格。

实际运行还受工作量分布影响。线程块太少、不同块执行时间不一致，以及最后一轮只剩少量工作，都可能使实际占用率低于理论值。若每轮能容纳 $B_{\mathrm{res}} N_{\mathrm{SM}}$ 个线程块，但总块数不能被该数量整除，末轮便无法像满负载阶段那样填满设备。报告中的各项平均值还可能使

用不同的活跃周期口径，因此需要同时检查 SM 活动和整个时间线。即使线程束槽位已占满，边界线程束中的部分线程也可能没有有效输出。

例如，若设备同时能容纳 264 个线程块，总工作量原本包含 1584 块，则可以形成 6 轮满负载执行。若粗化后块数变为 396，就形成一轮完整工作和一轮只有 132 块的尾部工作。假定各块耗时相同、分配理想，按两个完整轮次的容量计算，槽位利用比例仅为 $396/(2\times264) = 75\%$。这个比例描述网格规模与驻留容量不匹配带来的尾部损失；粗化还会改变每块耗时和资源需求，因此它不能直接预测最终加速比。

5.2 从驻留到就绪，再到实际发射

线程束调度器维护一组驻留线程束。每周期先检查哪些线程束的下一条指令已具备执行条件，再从这些就绪线程束中选择可发射者。因此，调度报告中的**驻留线程束**（*active warps*）、**就绪线程束**（*eligible warps*）和**已发射线程束**（*issued warps*）分别对应候选集合、当前具备条件的子集和实际发射行为。

图5给出的例子中，每调度器的理论上限为 12 个线程束，平均实际驻留 11.61 个，平均就绪却只有 0.75 个，平均每周期发射约 0.46 个线程束。没有就绪线程束的周期约占 53.89%。驻留状态已经接近满额，主要问题是很多线程束同时在等待，调度器缺少可以立即发射的工作。

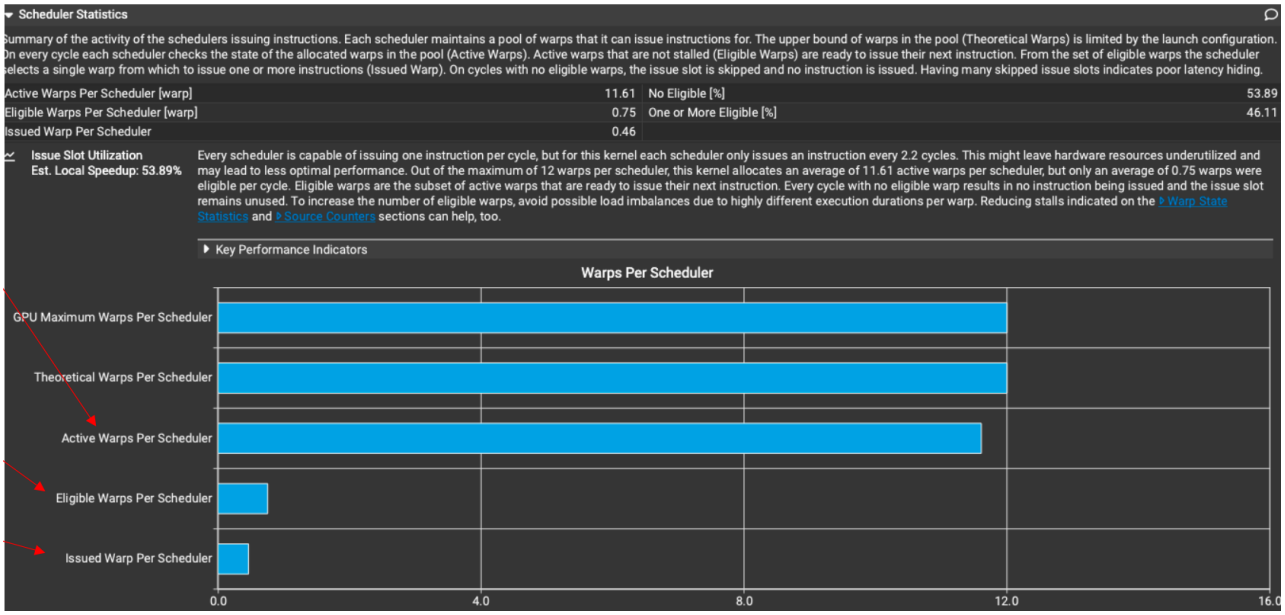


图 5 每调度器的理论、驻留、就绪和发射线程束统计。平均驻留数量接近上限，而就绪与发射数量明显较低，说明需要进一步检查等待原因。

平均每周期发射 0.46 个线程束，可理解为调度器平均约每 $1/0.46 \approx 2.17$ 周期发射一次。

报告中的“每条发射指令对应的线程束周期”还会把各驻留线程束经历的周期汇总，示例值约为 25.18，其中包含被选中、就绪但未被选中以及停顿等状态。两者的分母不同：前者看调度器整体的发射节奏，后者反映每次指令发射所对应的线程束周期。粗略地，11.61/0.46 ≈ 25.2，可以帮助理解两个数字为何处于不同数量级。

如果驻留数量很低，应先检查线程块规模、寄存器、共享内存和网格大小；如果驻留充足而就绪很少，就应检查数据依赖、存储等待与同步；如果就绪线程束较多而目标流水线已经持续忙碌，则要考虑其接收能力和指令组合。这一顺序能把占用率与真正的发射瓶颈联系起来。

5.3 线程束停顿与延迟隐藏

线程束状态统计（Warp State Statistics）把平均线程束周期分配到被选中、就绪但未被选中以及各类停顿状态。**记分板**(*scoreboard*)跟踪指令依赖是否已经满足：如果下一条指令需要某个尚未返回的加载结果，该线程束就必须等待。其他线程束仍可继续执行，因此某类停顿存在，并不自动说明整个 SM 同时空闲；应先结合调度报告确认是否有大量未使用的发射机会。

表6列出与矩阵乘法诊断最相关的几类状态。对应硬件通路的名称较具体，尤其需要区分等待数据返回与等待队列空位。

表 6 常见线程束等待状态及其解释

状态	正在等待什么	可进一步检查的内容
长记分板等待（Long Scoreboard）	L1 / 纹理路径操作产生的数据依赖，包括全局、局部、纹理或表面访问	请求合并、缓存命中、加载到使用之间的独立工作
短记分板等待（Short Scoreboard）	非 L1 / 纹理路径的相关操作依赖；共享读取是常见原因	共享布局、存储体冲突及共享数据的消费位置
存储输入输出节流（MIO Throttle）	对应指令队列尚无空位	共享加载数量、可否使用较宽加载，以及相关通路压力
数学流水线节流（Math Pipe Throttle）	所需数学流水线暂时不能接收下一条指令	指令是否集中在特定流水线，有无冗余运算或可调整的指令组合
同步屏障等待（Barrier）	同步要求的其他线程束尚未到达	块内工作是否均衡、加载与计算分工、同步的位置
未被选中（Not Selected）	线程束已经就绪，但本周期选择了其他线程束	结合实际发射率判断；存在这种状态说明调度器有可选工作

长记分板等待涉及的加载可能命中缓存，因而不能直接解释成全部时间都在等待显存。短记分板和存储输入输出通路还可能涉及特殊数学或动态分支等操作，具体应以热点指令为准。存储输入输出节流侧重队列接收能力；如果大量窄共享加载不断挤入同一入口，减少加载指令数量就可能有效，即使共享数据的总字节数保持不变。

在前述简单内核的示例中，每条发射指令对应约 25.18 个线程束周期，其中长记分板等待约占 20.6 周期，即约 81.8%。结合接近满额的驻留数量和较低的就绪数量，可以把重点放在加载依赖及其延迟隐藏上。增加更多驻留线程束受到硬件上限限制；提高数据复用、让加载更高效，或在等待结果期间提供其他独立计算，都可能改善当前状态。

同一线程束中的单一累加器也会形成连续依赖：下一次乘加需要前一次乘加的结果。使用多个输出累加器，可以提供多条独立链，让编译器有机会交错安排指令。这与线程束之间的切换共同隐藏延迟，但会增加寄存器占用。诊断时需要观察最终发射率与总时间，不能把某个占用率或某类停顿的单项变化作为唯一目标。

停顿统计还常以每条发射指令或采样次数作为分母。如果优化删除了很多指令，某种剩余等待的比例可能上升；执行时间仍可能下降，因为总工作量和主要队列压力已经减少。比较时应保持分母意识，同时观察绝对指令数量、运行时间和多个相关指标。

5.4 从源代码行追踪到机器指令

源码热点视图将设备代码行号与编译后的指令联系起来。虚拟指令集（Parallel Thread Execution，PTX）提供中间表示，目标架构机器指令（SASS）则反映具体 GPU 上使用的加载、存储、乘加和控制指令。源代码中的一行表达式可能展开为地址计算、加载、乘加与循环控制等多条指令；编译器还会进行展开、重排和寄存器分配，所以源码行与机器指令通常不一一对应。

使用 `-lineinfo` 编译后，可以按源码行查看执行指令数量、寄存器活跃情况与停顿采样，再沿 PTX 和 SASS 确认实际执行路径。例如，`LDG` 表示全局读取，`LDS` 表示共享读取，`STS` 表示共享写入，`FFMA` 表示单精度融合乘加。指令后缀还可能说明访问宽度或其他执行语义。

一个典型依赖链可以写成下面的机器指令结构。第一条从全局内存加载到寄存器 `R2`，第二条乘加使用 `R2`；地址寄存器和其他源寄存器在此前已经准备好。

代码 10 加载结果到消费指令的依赖示意

```
LDG.E R2, [R4];
FFMA  R8, R2, R6, R8;
```

若 `R2` 尚未准备好，停顿可能记录在等待使用它的乘加指令上。因此，热点落在乘加行并不说明乘加流水线本身一定是瓶颈；需要沿寄存器依赖回溯到生产该数据的加载。类似地，一条共享存储也可能因为等待前面的全局加载结果而呈现长记分板等待。

存活寄存器（*live registers*）表示在某个程序位置仍保存着后续需要使用的值的寄存器。它帮助理解哪些临时值同时占据状态，与整个内核最终分配的每线程寄存器数量相关，但两者

不是逐行相同的数值。查看向量化或线程粗化后的代码时，应同时确认宽读取是否生成、寄存器压力是否增大，以及占用率与局部内存访问是否发生变化。

一条完整的定位过程由这些观察逐步收敛：时间线确定主要耗时内核，吞吐与屋顶线提供性能上界，存储和调度报告定位繁忙通路或等待类型，源码热点与机器指令再确定可以修改的代码。修改之后，预期受影响的计数器应出现对应变化，并由正常计时验证实际收益。

6 矩阵乘法的比较与优化

6.1 三组设备上的测量结果

表7比较简单内核、共享分块、共享改进版、寄存器与共享内存联合分块，以及 cuBLAS。三组软件环境分别为 A40 上的 CUDA 12.4、L40S 上的 CUDA 12.2 和 B200 上的 CUDA 12.8。各组使用相同目标工作量，组内加速比以简单内核为基准。

表 7 矩阵乘法的独立计时、吞吐率与峰值效率

设备	实现	时间（ms）	GFLOP/s	加速比	效率
A40	简单内核	9.92	677	1.00	1.81%
	共享分块	3.34	2 011	2.97	5.38%
	共享改进版	2.29	2 935	4.33	7.85%
	联合分块	1.90	3 545	5.22	9.48%
	cuBLAS	0.30	22 026	33	58.90%
L40S	简单内核	3.47	1 937	1.00	2.96%
	共享分块	1.92	3 498	1.81	5.35%
	共享改进版	1.31	5 138	2.65	7.85%
	联合分块	1.06	6 354	3.27	9.71%
	cuBLAS	0.18	36 631	19.28	55.98%
B200	简单内核	4.42	1 520	1.00	3.21%
	共享分块	1.35	4 980	3.27	10.53%
	共享改进版	1.07	6 256	4.13	13.22%
	联合分块	1.03	6 514	4.30	13.77%
	cuBLAS	0.17	39 907	26	84.35%

时间、吞吐率和加速比各列经过独立舍入，因而用显示的两位小数时间重新计算，可能与其他列略有出入。将表中各行时间与吞吐率相乘，可以回推出共同目标工作量约为 $F = 6.72 \times 10^9$ 次浮点运算。这个工作量适合用来检查理论时间；仅由表格和启动配置，尚不足以唯一确定 M, N, K。

表8按 $t_{\min} = F/P_{\text{peak}}$ 统一计算纯计算下界。A40 的峰值分母采用提升时钟条件，未必等于整个测量区间内可持续达到的频率；L40S 与 B200 的实验分别将图形时钟锁定为 1800

MHz 和 1400 MHz。真实性能还受访存及指令通路约束，因此计算下界只用于建立参照。

表 8 指定峰值口径下的计算时间下界

设备	峰值（GFLOP/s）	峰值与实验时钟口径	$t_{\min}$（ms）
A40	37 400	提升时钟条件下的理论峰值	0.180
L40S	65 434	实验锁定 1800 MHz	0.103
B200	47 309	实验锁定 1400 MHz	0.142

例如，L40S 的计算过程为 $6.72 \times 10^9/(65\,434 \times 10^9)$ 秒，即约 0.1027 毫秒。相同数据下，简单内核相对于该纯计算下界的比值约为 $3.47/0.1027 \approx 33.8$。这样可以同时核对运算量、时间单位和乘加计数，避免把一次乘加在不同位置分别算作一次和两次浮点运算。

时钟控制命令使用设备编号选择目标 GPU，`-lgc` 的两个数值分别限定最小和最大图形时钟。以下示例将设备 0 锁定为 1800 MHz，最后一条恢复默认图形时钟控制。设置需要相应系统权限；B200 对应实验将数值换为 1400,1400。

代码 11 实验中的图形时钟控制示例

```
sudo nvidia-smi -i 0 -lgc 1800,1800
sudo nvidia-smi -i 0 -rgc
```

共享改进版在三组设备上都快于原共享版本，但收益幅度不同；联合分块又继续改善了时间。cuBLAS 仍明显更快，说明减少全局读取只是优化链条中的一部分，还需要兼顾分块形状、寄存器复用、指令调度和各流水线的工作分配。

跨设备效率比较还受到分母影响。B200 上 cuBLAS 的吞吐率约为 39.91 TFLOP/s，略高于 L40S 的 36.63 TFLOP/s；但两组使用的峰值分母分别约为 47.31 与 65.43 TFLOP/s，因此效率分别为 84.35% 与 55.98%。效率差异同时反映实现与所选峰值口径，不能只据此概括设备的所有性能特征。简单内核在 L40S 上又比 B200 更快，也说明具体实现与资源匹配决定最终次序。

6.2 简单内核到共享分块：瓶颈如何改变

图6展示不同矩阵乘法实现的屋顶线位置。简单内核与共享分块的横坐标次序，反映的是报告所用存储层级上的实测算术强度。共享分块的运行更快，却可能因为设备内存读取增加而向左移动；联合分块和库实现则在不同程度上改善复用与执行效率。图中不同水平峰值线对应不同精度口径，判断单精度点时应使用匹配的上界。

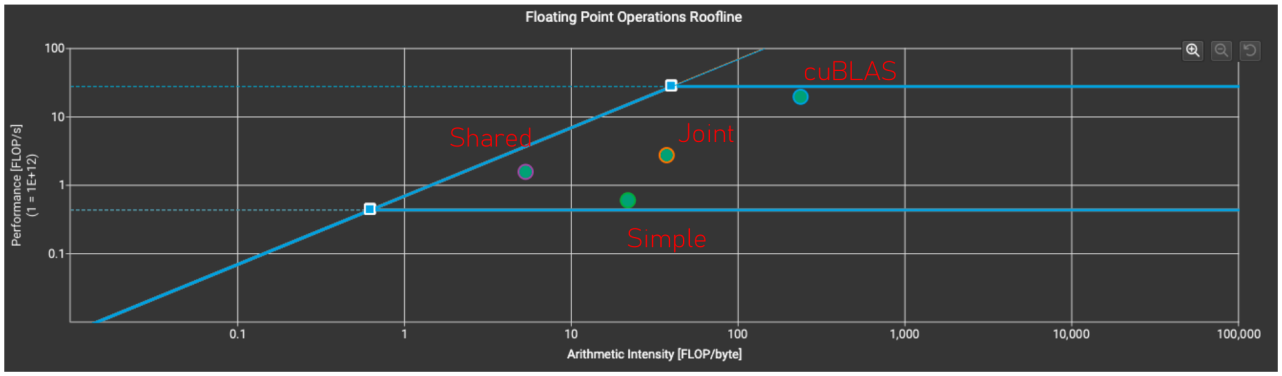


图 6 简单、共享、联合分块与库实现的屋顶线位置。纵坐标为浮点吞吐率，横坐标采用报告层级的实测算术强度；性能提升并不要求横坐标始终增大。

表9给出简单与共享内核的几项报告数据。全局加载请求由约 210.00 百万降至 13.22 百万，接近 16 倍的减少，与 $T = 16$ 分块的复用机制相符。与此同时，共享读取请求达到约 211.76 百万，说明原有的全局读取需求有很大部分转成了片上读取工作。

表 9 简单内核与共享分块的存储和占用率比较

指标	简单内核	共享分块
全局存储空间指令（百万）	210.07	13.29
全局加载请求（百万）	210.00	13.22
共享加载请求（百万）	0	211.76
L1 / 纹理缓存命中率	48.53%	11.21%
L2 命中率	98.19%	52.08%
L2 向 L1 返回的数据（GB）	9.11	1.69
设备内存读取量	290.32 MB	1.24 GB
理论占用率	100%	100%
实际占用率	96.86%	97.90%

简单内核的重复请求很多，但大量请求命中 L2；共享分块的逻辑请求减少后，访问顺序和复用在缓存中的表现也发生改变，L2 命中率降至约 52%。最终，L2 向 L1 返回的数据仍减少，设备内存读取却从约 290 MB 增加到约 1.24 GB。这些数字分别描述不同层级，因此能够同时成立。

共享内核的平均每请求扇区数约为 4.40，报告还显示额外扇区，提示部分全局访问有进

一步检查的空间。分析时要代入活跃线程数、每线程寄度、行跨度与对齐；不同内核的请求可能具有不同的广播与空间分布，单独比较平均扇区数并不足以确定具体代码问题。

两种内核的理论占用率都为 100%，实际占用率也都接近 97%，所以共享版本的收益并非来自显著增加驻留线程束。更明显的变化是等待从大量全局加载依赖转向共享访问和存储输入输出队列压力。共享版本的汇总资源利用率接近 98% 时，主要贡献还可以来自加载 / 存储相关通路；这正是前面区分资源汇总与浮点峰值效率的实际例子。

这些报告中的时钟与采集条件可能与表7的独立计时不同。报告比较用于验证硬件机制，最终加速幅度则回到同一正常计时条件下测量。

6.3 以向量读取缓解共享指令队列压力

共享内核的累加循环每步执行 `As[ty][k]` 与 `Bs[k][tx]` 两次共享读取，再完成一次乘加。对固定线程，连续四个 k 对应的 A 数据位于同一行相邻位置；相应的 B 数据沿列分布，相邻地址间隔 T 个元素。因此，可以把每线程连续的四个 A 元素读成一个向量，B 仍保留四次标量读取。

为了明确数据布局和对齐条件，可将 A 的共享子块表示为自然 16 字节对齐的 `float4` 数组。令 `As4[y][g]` 的四个分量依次保存原来的 A 子块元素 $A^s_{y,4g},\dots,A^s_{y,4g+3}$，并要求 T 为正的 4 的倍数。加载阶段使用对应的全局标量地址填入这些分量，在全部线程完成块内同步后，再执行代码12所示的累加函数。该函数只表达消费端的向量读取机制，外层分块加载、边界补零及覆盖前同步仍按共享内核的规则处理。

代码 12 对自然对齐的共享四元组进行向量读取

```
template<int T>
__device__ float accumulate_vector_tile(
    const float4 (&As4)[T][T / 4],
    const float (&Bs)[T][T],
    int ty, int tx, float sum) {
  static_assert(T > 0 && T % 4 == 0,
                "T must be a positive multiple of four");
  #pragma unroll
  for (int g = 0; g < T / 4; ++g) {
    float4 a = As4[ty][g];
    int k = 4 * g;
    sum += a.x * Bs[k + 0][tx];
    sum += a.y * Bs[k + 1][tx];
    sum += a.z * Bs[k + 2][tx];
    sum += a.w * Bs[k + 3][tx];
  }
  return sum;
}
```

这个表示让向量对象的类型和对齐条件明确，避免把任意未对齐地址直接解释成宽向量。每个线程仍需要相同的输入数值，并按相同的 k 顺序累加；改变的是这些值进入寄存器时使用的读取指令宽度。是否最终生成一条宽共享加载，仍由编译结果决定，需要检查机器指令。

```
55 00007feb 47264960      LDS R30, [R18.X4+0x400]
56 00007feb 47264970      LDS.128 R12, [R24]
57 00007feb 47264980      LDS R29, [R18.X4+0x440]
58 00007feb 47264990      LDS R34, [R18.X4+0x480]
59 00007feb 472649a0      LDS R32, [R18.X4+0x4c0]
60 00007feb 472649b0      LDS R31, [R18.X4+0x500]
61 00007feb 472649c0      LDS.128 R4, [R24+0x10]
62 00007feb 472649d0      LDS R28, [R18.X4+0x540]
63 00007feb 472649e0      LDS R25, [R18.X4+0x580]
64 00007feb 472649f0      LDS R26, [R18.X4+0x5c0]
65 00007feb 47264a00      LDS R27, [R18.X4+0x600]
```

图 7 共享向量读取对应的机器指令片段。宽指令 LDS.128 与保留的标量 LDS 同时存在，对应向量化的连续读取和其他标量读取。

以连续四次乘加为一个计数单位，原实现需要四次 A 标量读取与四次 B 标量读取，共 8 次共享加载；向量化后需要一次 A 宽读取与四次 B 标量读取，共 5 次。因此，理想共享加载请求的下降比例为

$$r_{\text{load}} = \frac{8-5}{8} = 37.5\%. \tag{7}$$

单看 A 的读取指令，数量减少到四分之一；把 A、B 都计入，整体减少到八分之五。四次乘加本身保持不变，所需标量数据量也保持不变。LDS.128 中的 128 位是每线程的读取宽度，不能与一轮共享服务的 128 字节混用。

6.4 由指标变化验证优化机制

表10给出原共享内核和改进版的比较。共享加载从约 211.76 百万降到 132.35 百万，下降 37.50%，与式(7)一致。共享存储请求保持约 13.24 百万，因此计入写入后的共享总指令下降比例较小，为约 35.29%。

表 10 共享读取向量化前后的报告变化

指标	共享分块	共享改进版	相对变化
共享加载请求（百万）	211.76	132.35	−37.50%
共享存储请求（百万）	13.24	13.24	0%
共享总指令（百万）	225.00	145.59	−35.29%
全局存储空间指令（百万）	13.29	13.29	0%
设备内存读取量（GB）	1.24	1.23	约 −1.34%

表中的数据量经过舍入，相对变化可能由更高精度的报告计数计算。例如 1.24 GB 与 1.23 GB 的显示值不足以重现 −1.34% 的报告变化，应将其解释为小幅下降。

全局加载请求与全局指令基本保持不变，设备内存读取量也只有小幅波动，这与局部修改共享读取方式的范围相符。图8的指令组合显示共享加载显著减少，浮点乘加数量保持不变；总执行指令下降约 15.32%。因此，这次改进的主要证据是减少共享加载指令与队列压力，而非显著改变设备内存流量。

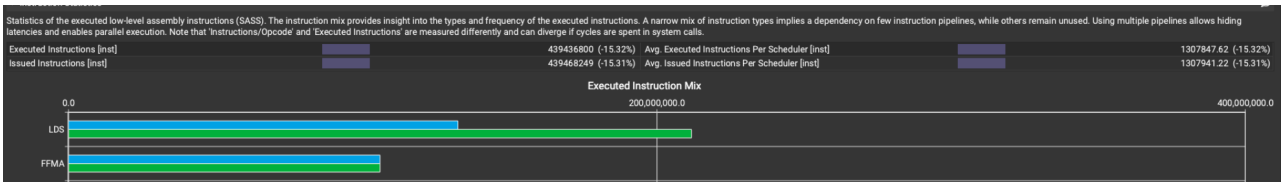


图 8 共享改进版与原共享版本的指令组合比较。上方共享加载条带缩短，下方单精度乘加条带基本重合，说明算术工作保留而读取指令减少。

线程束状态比较也与这个机制相符。图9中，存储输入输出节流的平均周期约从 24.4 降到 16.0；部分其他等待没有同步下降，长记分板等待还略有增加。这并不破坏当前解释，因为优化针对的是主要共享指令压力，总执行指令和主要等待已经减少。

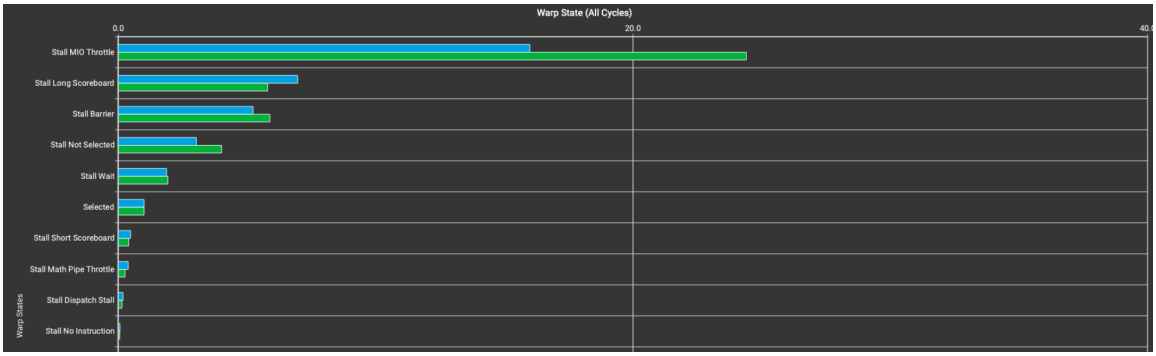


图 9 共享改进版与原共享版本的线程束状态比较。蓝色对应改进版，绿色对应原共享版本；主要变化是存储输入输出节流降低，其他等待不必按相同比例变化。

最后回到正常 CUDA 事件计时。A40 上耗时从 3.34 毫秒降至 2.29 毫秒，相对原共享版本达到约 1.46 倍加速；L40S 上从 1.92 降至 1.31 毫秒，约为 1.47 倍；B200 上从 1.35 降至 1.07 毫秒，约为 1.26 倍。总时间没有按共享加载请求的比例直接下降，因为全局读取、共享写入、乘加、同步及其他指令仍然存在，减少某一类请求也不等于消除了所有等待。

进一步优化时，多余扇区提示检查线程映射与对齐，片上加载繁忙提示考虑寄存器复用，寄存器用量压低驻留数量则提示调整粗化程度或分块形状。修改后应核对相关计数器、数值结果及同条件计时。

性能诊断的关键关系

诊断应同时联系工作量、指令数、各层流量、驻留资源与发射状态，并用总时间判断修改的收益。